

# LEARNING RESOURCE GUIDE

Generative AI for Beginners | Professional Participant Workbook

|         |                                                |
|---------|------------------------------------------------|
| Section | Section 2: Neural Networks - The Foundation    |
| Lecture | Lecture 2.2: How Neural Networks Work - Part 2 |

## Learning Objectives

By the end of this lecture and its accompanying guide, you will be able to:

- Explain how information flows through a neural network from input to output (forward propagation).
- Describe in depth how weights and biases determine a neuron's output and why they are the primary learning targets.
- Distinguish between the major activation functions (sigmoid, tanh, ReLU, GELU) and explain why ReLU dominates modern architectures.
- Articulate what forward propagation produces and how it relates to the model's prediction.
- Understand why the composition of many simple linear transformations with non-linear activations produces complex representational power.

## Introduction

In Part 1, we examined the individual neuron and the layered structure of neural networks. Now we focus on the dynamics: what actually happens when data enters the network and travels through each layer to produce an output. This process — called forward propagation — is the inference step that runs every time you send a message to ChatGPT, generate an image in Midjourney, or use any AI-powered application.

Forward propagation is conceptually straightforward: data enters the input layer as numbers, is multiplied by weights, summed with biases, passed through activation functions, and this process repeats layer by layer until the output layer produces a final result. But the consequences of this simple repeated operation are profound — a sufficiently deep network applying this process over many layers can learn to represent extraordinarily complex functions, from human faces to natural language to molecular structures.

Understanding forward propagation also lays the groundwork for understanding training (backpropagation), which runs this process in reverse to compute how each weight should be adjusted to reduce prediction error. Part 1 gave you the structure; Part 2 gives you the dynamics of inference; Part 3 (Training Neural Networks) will give you the dynamics of learning.

## Core Concepts Explained

### Forward Propagation

**Definition:** Forward propagation is the process by which input data flows through a neural network, layer by layer, from the input layer to the output layer. At each layer, each neuron computes a weighted sum of its inputs, adds a bias, and applies an activation function. The output of each layer becomes the input to the next.

**Business relevance:** Forward propagation is the inference step — the computation that runs when a model makes a prediction or generates an output. Understanding it clarifies what is happening computationally when an AI tool responds to your input, and why inference time and cost depend on network depth and width.

**Practitioner relevance:** For professionals deploying AI models, forward propagation latency (how long inference takes) is a key operational metric. Deeper and wider networks require more computational steps during forward propagation, affecting response time and inference costs in production systems.

**Real-world example:** When you send a prompt to a large language model, forward propagation runs that prompt through tens or hundreds of transformer layers — each layer performing matrix multiplications (the neural network equivalent of weighted sums) and activation functions across billions of parameters — to produce probability distributions over possible next tokens. This happens in seconds on specialised hardware.

**Risks or limitations:** The computational cost of forward propagation scales with model size and context length. Very large models with long context windows can have prohibitive inference costs for high-volume applications, requiring architectural optimisations or model distillation.

## Weights and Biases in Depth

**Definition:** Weights are the parameters that determine how strongly each input signal influences each neuron's output. Biases shift the activation threshold, allowing neurons to activate even when all inputs are zero or to remain inactive even when inputs are non-zero.

**Business relevance:** Weights and biases are where the knowledge is. When a model is said to have learned to write code, recognise faces, or translate languages, what this means mechanically is that the weights and biases in the network have been optimised (through training) to produce correct outputs for those tasks. The model's capabilities are fully encoded in these numbers.

**Practitioner relevance:** Understanding that model knowledge is entirely encoded in weights and biases explains phenomena like model fine-tuning (adjusting a small subset of weights for a specific task), model distillation (training a smaller network to replicate the weight-encoded knowledge of a larger one), and model editing (directly modifying specific weights to change model behaviour).

**Real-world example:** When researchers say they have fine-tuned a model on medical data, they mean they have continued training an already-trained model on medical text, adjusting the existing weights and biases to better represent medical knowledge. The pre-trained weights provide a strong starting point; fine-tuning specialises them.

**Risks or limitations:** Weights encode everything the model has learned, including biases and problematic patterns from training data. There is no separate box of clean knowledge and a separate box of biased knowledge — the biases are woven into the same weights that encode the model's capabilities.

## Activation Functions — Sigmoid, Tanh, ReLU, GELU

**Definition:** Activation functions introduce non-linearity after each neuron's weighted sum. Sigmoid maps outputs to 0 to 1 (used for binary classification output layers). Tanh maps to negative 1 to 1. ReLU outputs zero for negatives, value for positives. GELU is a smooth approximation of ReLU used in transformer models.

**Business relevance:** The choice of activation function affects training speed, convergence, and model performance. ReLU became dominant because it addresses the vanishing gradient problem that plagued sigmoid and tanh (where gradients become too small to propagate effectively through many layers). GELU is the current standard in large language models.

**Practitioner relevance:** Knowing the standard activation functions allows professionals to read model architecture documentation and understand why certain choices were made. GELU in a model description

signals a modern transformer-based architecture; sigmoid in a classification output layer is expected and appropriate.

**Real-world example:** GPT models use GELU activation functions in their feed-forward network components. BERT uses GELU in its transformer layers. The choice of GELU over ReLU was made based on empirical evidence that GELU slightly improves performance on language tasks — a small architectural detail with meaningful downstream effects.

**Risks or limitations:** The dying ReLU problem occurs when neurons consistently receive negative pre-activation values and permanently output zero, becoming useless. Leaky ReLU and GELU partially address this by allowing small non-zero outputs for negative inputs.

## The Output Layer and Softmax

**Definition:** The output layer produces the network's final prediction. For classification tasks, a softmax function is often applied to convert the final layer's raw scores (logits) into a probability distribution that sums to 1 across all possible classes or tokens.

**Business relevance:** In language models, the output layer produces a probability distribution over the entire vocabulary (tens of thousands of tokens) at each generation step. The model selects the next token by sampling from this distribution — a process controlled by parameters like temperature that affect output diversity versus predictability.

**Practitioner relevance:** Understanding that language model outputs are probability distributions — not factual lookups — is the key to understanding both their fluency (they predict statistically coherent sequences) and their hallucination tendency (they predict probable text, not necessarily true text).

**Real-world example:** After processing the prompt The capital of France is, a language model's output layer produces a distribution where Paris has very high probability, but Lyon, Berlin, and London also have non-zero probability. Sampling deterministically selects Paris; sampling with high temperature might occasionally select a less likely token, producing more creative but potentially less accurate outputs.

## Practical Applications

---

### Understanding AI Response Latency

The time it takes for an AI model to respond to a prompt is largely determined by the number of forward propagation steps required. More layers, more parameters, and longer sequences all increase computation. This is why smaller, distilled models are often preferred for latency-sensitive production applications.

### Temperature and Sampling in Language Models

The probability distribution produced by the output layer's softmax can be sampled deterministically (always pick the most probable token) or stochastically (sample proportionally to probability). Temperature scaling adjusts the sharpness of this distribution — low temperature produces predictable, conservative outputs; high temperature produces creative, diverse but potentially less accurate outputs.

### Model Fine-tuning and Transfer Learning

Because the model's knowledge is encoded in weights and biases, fine-tuning consists of continuing to train a pre-trained model on a specific dataset, adjusting the weights to specialise the model for the target domain. This is far more efficient than training from scratch because the pre-trained weights already encode general linguistic or visual knowledge.

## Best Practices

- Use temperature settings intentionally. In deployed language model applications, low temperature produces consistent, predictable outputs appropriate for factual tasks. Higher temperature suits creative tasks. Match the temperature to the task requirements.
- Consider inference cost when selecting models. Larger models with more layers and parameters require more computation per inference call. For high-volume applications, smaller fine-tuned models that meet quality thresholds are often more economical.
- Interpret model outputs as probability distributions, not facts. The softmax output is a probability distribution over possible next tokens — it represents statistical plausibility, not factual correctness. This is a fundamental property of how the output layer works, not a quality issue with specific models.
- Evaluate model distillation for deployment efficiency. Model distillation trains a smaller model to replicate the output behaviour of a larger model. The distilled model can approach the larger model's performance at a fraction of the compute cost — a critical technique for practical deployment.

## Common Mistakes and Misconceptions

**Misconception:** Assuming that a model with high softmax probability for an output is factually correct.

**Correct approach:** High softmax probability means the output is statistically common in training data, not that it is factually correct. A model can be highly confident about a false statement if that false statement appeared frequently in training data.

**Misconception:** Treating forward propagation as understanding — the model understands language.

**Correct approach:** Forward propagation is a mathematical transformation of numerical representations. The model performs highly effective pattern matching through layered matrix operations, but this is not understanding in the human cognitive sense. This distinction matters when deploying models in high-stakes contexts.

**Misconception:** Assuming all layers in a network learn the same types of features.

**Correct approach:** Early layers learn low-level features (edges, phonemes, character patterns); middle layers learn intermediate features; late layers learn high-level, task-relevant abstractions. This hierarchical specialisation is a key source of deep network effectiveness.

## Decision Framework: Choosing Between Sampling Strategies for Language Model Outputs

Use this framework to select the appropriate sampling strategy when configuring a language model for a specific application.

| Use Case                     | Recommended Temperature    | Sampling Strategy             | Rationale                                     |
|------------------------------|----------------------------|-------------------------------|-----------------------------------------------|
| Factual Q&A, data extraction | Low (0.0 to 0.3)           | Greedy / near-deterministic   | Maximise consistency and accuracy             |
| Code generation              | Low to medium (0.1 to 0.5) | Near-deterministic            | Correctness matters more than variety         |
| Summarisation                | Low to medium (0.2 to 0.5) | Sampling with low temperature | Consistent with slight paraphrasing variation |

|                       |                             |                                 |                                     |
|-----------------------|-----------------------------|---------------------------------|-------------------------------------|
| Creative writing      | Medium to high (0.7 to 1.2) | Temperature sampling            | Diversity and novelty are valued    |
| Brainstorming         | High (0.8 to 1.5)           | Temperature sampling with top-p | Maximum diversity of suggestions    |
| Customer service chat | Low (0.2 to 0.4)            | Near-deterministic              | Consistency and safety of responses |

**How to use this:** Most commercial APIs expose temperature and top-p (nucleus sampling) parameters. Start with the recommended temperature for your use case and adjust based on output quality evaluation.

## Knowledge Check

Test your understanding before moving on. Answers are shown below each question.

### 1. Forward propagation in a neural network refers to which process?

- A. Adjusting weights backwards from the output to input based on prediction error.
- B. The flow of data from input through each layer to the output, computing weighted sums and activations.
- C. Loading model weights from storage into memory for inference.
- D. The process of evaluating model performance on a held-out test set.

**Answer:** B. The flow of data from input through each layer to the output, computing weighted sums and activations.

**Why:** Forward propagation is the inference process — data enters the input layer and is transformed layer by layer through weighted sums and activation functions until the output layer produces a result.

### 2. A language model generates the phrase *The Eiffel Tower is located in Berlin with very high confidence*.

What does this indicate about the model?

- A. The model has a factual knowledge database that contains an error.
- B. The softmax output assigned high probability to Berlin, which may reflect a pattern in training data — but high probability does not guarantee factual correctness.
- C. The model's weights were incorrectly initialised.
- D. Temperature was set too high, causing random token selection.

**Answer:** B. The softmax output assigned high probability to Berlin, which may reflect a pattern in training data — but high probability does not guarantee factual correctness.

**Why:** Language models predict statistically probable token sequences. High confidence in a factually incorrect statement indicates the model has learned to associate those tokens — not that it has verified the fact.

### 3. A team deploys a large language model for a high-volume customer service application and finds inference costs prohibitive. Which approach best addresses this while maintaining output quality?

- A. Increase the temperature setting to reduce computation.
- B. Use model distillation to train a smaller model to replicate the large model's behaviour at lower inference cost.
- C. Reduce the number of training epochs.
- D. Remove hidden layers from the model to reduce forward propagation steps.

**Answer:** B. Use model distillation to train a smaller model to replicate the large model's behaviour at lower inference cost.

**Why:** Model distillation produces a smaller model that approximates the output behaviour of a larger one — often approaching the larger model's performance at a fraction of the inference cost, making it practical for high-volume deployment.

## Reflection Questions

---

1. The concept of temperature in language model sampling — trading predictability for creativity — has analogies in professional decision-making. When do you want high predictability versus creative exploration in your work context?
2. Forward propagation computes a transformation of inputs into outputs without any storage or retrieval of facts. How does this change how you think about using AI for research or fact-finding tasks?
3. Model weights encode both capabilities and biases from training data simultaneously. Given this, what verification and oversight processes would you put in place for an AI system making consequential recommendations in your organisation?
4. Inference cost scales with model size and context length. How would you approach the trade-off between output quality and cost for an AI application you might deploy?

## Key Takeaways

---

- Forward propagation is the inference process — data flows from input to output through successive weighted sums and activation functions across all layers.
- All model knowledge — capabilities and biases alike — is encoded in the weights and biases adjusted during training.
- Activation functions introduce non-linearity that enables deep networks to learn complex patterns; ReLU and GELU are the dominant choices in modern architectures.
- The output layer's softmax produces a probability distribution over possible outputs — representing statistical plausibility, not factual correctness.
- Temperature controls the sharpness of the output probability distribution, trading predictability (low temperature) for diversity and creativity (high temperature).
- Inference cost scales with model depth, width, and context length — driving practical trade-offs between capability and deployment feasibility.
- Model distillation is the primary technique for making large model capabilities accessible at lower inference cost.

## Recommended Next Actions

---

5. Experiment with temperature settings in a language model API or chat interface. Ask the same question at temperature 0.1 and temperature 1.0 and compare the outputs.
6. For one AI application you use at work, estimate: how deep might the model be, and what does that imply about the computation happening during each response?
7. Add to your glossary: Forward Propagation, Softmax, Logits, Temperature, Model Distillation, Fine-tuning, Transfer Learning.
8. Write a one-paragraph explanation of why a language model can be highly confident about a factually incorrect answer, suitable for explaining to a non-technical colleague.